

Stan Pierre
March 27, 2026

Week 8 Report

Since the last report, I shifted focus from building features separately to integrating everything directly into the main app so the system works as one cohesive pipeline. Instead of treating detection and formation logic as isolated pieces, I started connecting them into the actual user workflow.

What I actually ended up doing was working on integrating the detection system into the main application. I got pretty far with this and now have the core pieces running together instead of as separate tests. As I was doing that a lot of UI issues came up so I spent time fixing layout problems and making sure the flow makes sense when adding and processing videos. I also worked on video management features. This includes being able to add videos into the system and organize them in a way that makes them easy for processing later. The goal here is to make it easy to go from uploading a clip to eventually running detection and formation tagging without extra steps. This part is still in progress but at least it is there now.

The plan moving forward is to finish cleaning up the UI and complete the video management system so clips can be easily added and used by the system. After that, I will reconnect the detection and formation logic fully into this and continue improving accuracy, like around the overlapping player issue. Also continuous testing on formation detection on different angles and POVS.

Time Tracking

Week 8 (13 hours)

- Detection system into main app: 5 hours
- UI fixes and layout improvements: 3 hours
- Building video upload and management features: 3 hours
- Testing and debugging: 2 hours

Stan Pierre
March 13, 2026

Week 7 Report

Since the last report, I continued working on the formation recognition logic, but ran into some significant challenges with the clustering algorithm. The jersey color detection is working better after adjusting the thresholds, and the field ROI filtering continues to keep most refs and sideline personnel out of the results. However, the core detection still has issues that need to be solved before I can move forward.

What I actually ended up doing was testing the offensive line clustering on different clips and realizing the detection breaks down when players are too close together. If two offensive linemen are standing right next to each other or slightly overlapping from the camera angle, the detection system blobs them into one bounding box instead of separating them. This throws off the entire formation analysis because it thinks there are only four linemen instead of five, or misidentifies positions. I tried adjusting the confidence threshold and the clustering logic, but it's a problem with how the model groups nearby objects.

I also realized that the jersey color detection won't work universally because every team wears different colors. What works for one game including detecting dark vs light it completely fails when both teams wear colored jerseys or when one team wears an alternate uniform. My current thinking is to build a GUI setup process for each game. Before processing any clips, the system would ask, what color is your team wearing, what color is the opponent wearing, and let the user draw or select the field boundaries on a sample frame. Once that's configured for a game, the rest of the processing can be automated using those parameters.

The plan moving forward is to fix the clustering issue by either improving how the model handles overlapping players or by implementing a manual correction step where the user can split merged detections. After that, I'll build the game setup GUI so the color and field configuration can be customized per game instead of hardcoded. Once those pieces are solid, I can move on to actually classifying formations and automating the video workflow.

Time Tracking

Week 7 (13 hours)

- Testing clustering on different clips: 3.5 hours
- Debugging overlapping player detection issues: 3 hours
- Researching solutions for blob separation: 2 hours
- Planning GUI setup for game setup: 2.5 hours
- Testing jersey color detection across different uniform combinations: 2 hours

Stan Pierre

March 6, 2026

Week 6 Report

Since the last report, I continued improving the detection system and started working on the formation breakdown logic. The player detection is more stable now. With the field region filtering is working well and cutting out most of the sideline noise like refs and coaches. I also added jersey color detection to try and tell the difference between the two teams, though that still needs some tuning since it's just using colors right now and sometimes misclassifies players.

What I actually ended up doing was building out the formation recognition piece. Right now the system can detect players and group them by team, but it still can't identify specific formations like "Doubles Right" or "Trips Left." I'm working on logic to find the offensive line first like identifying the center based on clustering, then mapping other positions relative to that. The idea is to look at relationships in space, how many receivers are on each side, where the running backs are positioned, and whether there's tight end presence. Once I can reliably identify the formation from a single frame, the next step is automated video processing.

The plan moving forward is to get consistent formation recognition working across different camera angles and field orientations. After that, I want to automate the entire video workflow - you upload a clip, it extracts the pre-snap frame, runs detection, identifies the formation, and tags it automatically. That would eliminate most of the manual data entry and make the whole system actually usable for analyzing full games.

Time Tracking

Week 6 (14 hours)

- Field region filtering boundary logic: 3 hours
- Jersey color detection: 2.5 hours
- Formation recognition initial logic: 4 hours
- Offensive line clustering and center identification: 3 hours
- Testing clips and debugging: 1.5 hours

Stan Pierre
February 20 2026

Week 4 Report

Since the last report, I focused on improving player detection and getting a more reliable model running. I was able to fix the earlier compatibility problems by setting up a separate Python 3.11 environment and installing YOLOv8 from Ultralytics. That allowed me to move away from OpenCV's built in person detector and use a stronger object detection model.

What I actually ended up doing was getting the model to draw colored bounding boxes around detect players and then return coordinates for each one. Detection accuracy is noticeably better than before. It is consistently identifying players during pre snap frames. There are still issues. In some clips, grouping is off and the model occasionally includes referees or someone on the sideline. It also does not understand positions. Right now it only detects "person." I am storing the bounding boxes so I can begin figuring out spatial relationships between players instead of relying only on the detection.

The next step is to change from object detection to formation logic. I need to figure out the offensive line clutter and find the center, and then find all other player positions relative to that point. I also need to make the field orientation so formations can be shown consistently across clips. After that, I will work on making sure there is consistency in detection results and filtering out non players using field region limits.

Time Tracking

Week 4 (12 hours)

- Setting up Python 3.11 virtual environment and installing YOLOv8: 2 hours
- YOLOv8 into GridIQ: 3 hours
- Running detection on clips and messing with the model: 3.5 hours
- Setting up bounding box visualization: 2 hours
- Testing spatial grouping logic for offensive line: 1.5 hours

Stan Pierre
February 13 2026

Week 3 Report

Since the last report, I planned to do more research on OpenCV and switch over to Catapult's video system after getting logged out of our previous one. I also planned to use OpenCV on a small number of simple test cases to see how well it works at different scales.

What I actually ended up doing was running OpenCV on real football screenshots to test player detection. I set up a proof of concept using pre-trained models to see if it could identify players on the field. The results were mixed. It detected people pretty well, but it couldn't tell the difference between players, refs, coaches, and people on the sideline. It also couldn't distinguish between the two teams since it just sees everyone as "person." I ran into a technical issue with Python 3.13 compatibility, so I had to work around that by switching to OpenCV's built-in models instead of YOLOv8.

Moving forward, I need to figure out how to make the computer vision more accurate. Right now it's detecting too many people who aren't actually players on the field. I'm thinking about adding jersey color detection so it can tell the difference between the two teams. One team wears dark jerseys and the other wears white, so that should be doable. I also need to filter out refs, coaches, and sideline personnel. My idea is to focus on a specific area of the field where the actual play happens and ignore everything outside that zone. Once I can reliably identify just the players, I can start working on recognizing formations based on where they're positioned.

Time Tracking

Week 3 (11.5 hours)

- Setting up OpenCV environment: 2 hours
- Testing player detection on screenshots: 7 hours
- Troubleshooting Python compatibility issues: 1.5 hour
- Research on pre-snap frame extraction: 1 hour

Stan Pierre
February 6 2026

Week 2 Report

Since the last report, I planned to do more research on OpenCV and switch over to Catapult's video system after getting logged out of our previous one. I also planned to use OpenCV on a small number of simple test cases to see how well it works at different scales.

What I actually ended up doing was testing OpenCV on a few basic tasks, like having it tell the difference between photos I give it. I've got it set up in my project now. I also set up my accounts for the other systems I'll need to use going forward.

I'm planning to integrate video into the database somehow, since we need a place to store it. The problem is that video takes up a lot of space, so I need to figure out a way to keep storage low without spending money. The video itself is also pretty inconsistent, which makes identifying players harder. My idea right now is to grab a screenshot right before the ball is snapped. That's when everything is still and the positions are the clearest. From there, I can compare frames to see where players are between shots. This should be more accurate and could actually make tagging work.

Time tracking

Week 2 (5 hours)

- Downloading libraries : 1 hour
- Download videos: .5 hour
- using catapult framework: 1 hour
- Test OpenCv: 2.5 hours